Day1Notes

Tutorial Notes for Students

These notes accompany the tutorials in **upstream-sdk/tutorials**. They are intended to help you understand the main ideas behind the Upstream platform before and during the notebook exercises.

- DSO Institute Site
- Main Upstream

The tutorials are easiest to follow when you first understand how the platform is organized in the UI, and then see how the SDK works with the same concepts in Python.

What You Should Learn

By the end of the tutorial, you should understand the following four concepts:

- a campaign is the project-level container for a monitoring effort
- a station is a specific monitoring site within a campaign
- a sensor defines what is being measured at a station
- a measurement is a time-stamped observation associated with a sensor

These ideas appear in both the UI and the SDK. The main difference is that the UI presents them interactively, while the SDK lets you work with them programmatically.

Begin With The UI

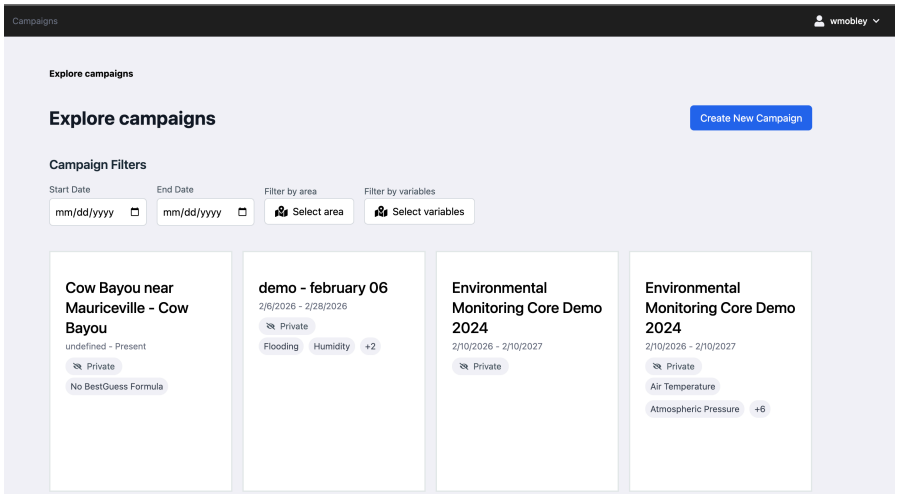
Before working through the notebooks, it is helpful to study the UI. The UI provides the clearest introduction to how Upstream organizes data and how the different parts of the platform relate to one another.

Campaigns

In the UI, a campaign represents the overall monitoring project.

What to learn:

- campaigns group together stations, sensors, uploads, and related project information
- a campaign gives you the top-level context for understanding the data
- campaign pages often summarize the scope of a project through counts or status indicators



What to notice in a screenshot:

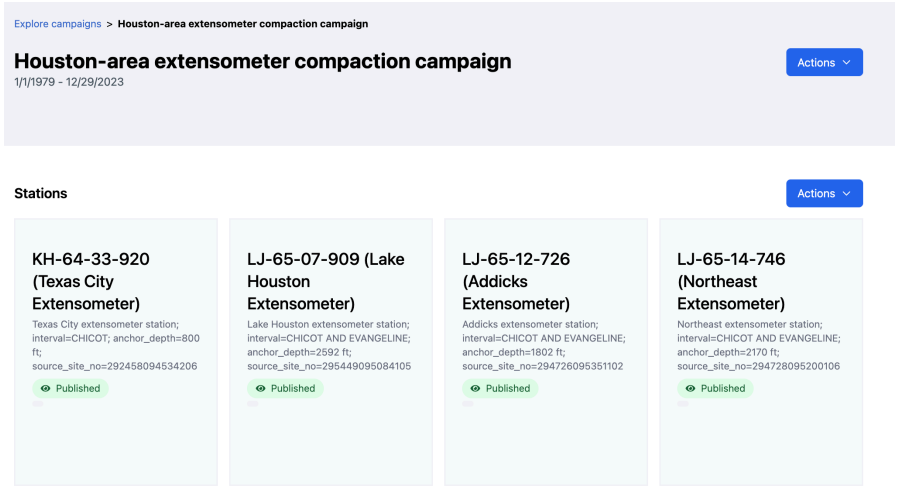
- the campaign name
- any summary information, such as station counts or sensor coverage
- navigation from the campaign to stations or other related records

Stations

A station represents a specific location or site within a campaign.

What to learn:

- stations connect the project structure to real monitoring locations
- station pages bring together metadata, uploads, and sensors
- many practical workflows begin at the station level



What to notice in a screenshot:

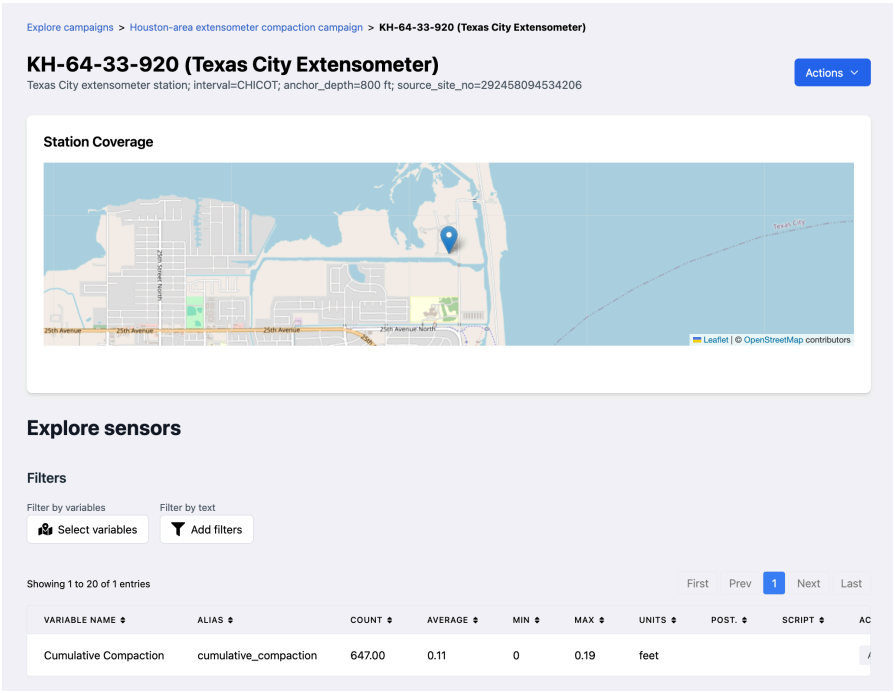
- the station name and metadata
- any map or geometry shown for the station
- the section where sensors, uploads, or related observations are displayed

Sensors

Sensors describe what is being measured at a station.

What to learn:

- sensors define the variables recorded at a monitoring site
- sensor information usually includes an alias, a variable name, and units
- understanding sensors is important because the uploaded measurement columns correspond to these definitions



What to notice in a screenshot:

- the sensor alias
- the variable name
- the units
- any summary statistics or details that help explain the meaning of the sensor

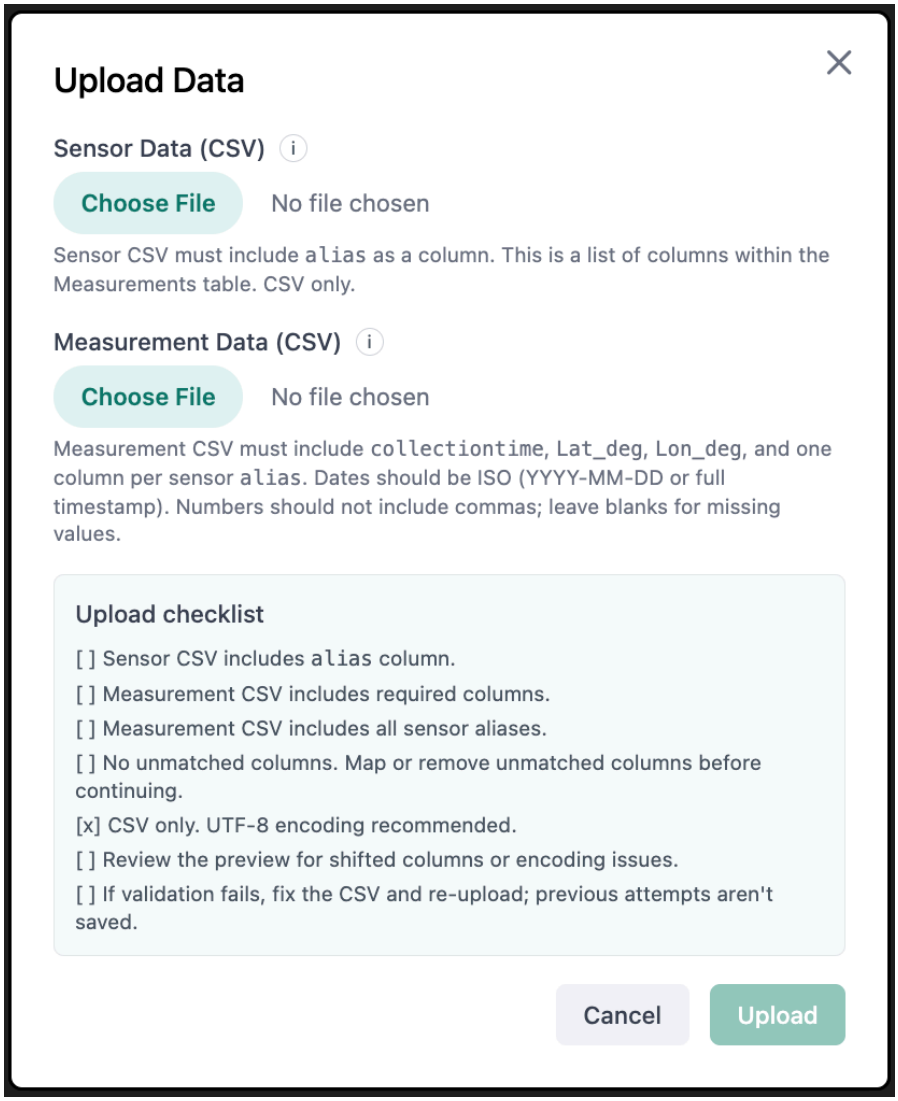
Data Upload

The upload interface shows how data enters the platform.

What to learn:

- uploads connect structured CSV files to a specific station

- one file describes sensors and another provides time-series measurements
- validation feedback helps identify problems before data is accepted



What to notice in a screenshot:

- the file selection area
- any instructions about expected CSV structure
- validation or success messages, if they are shown

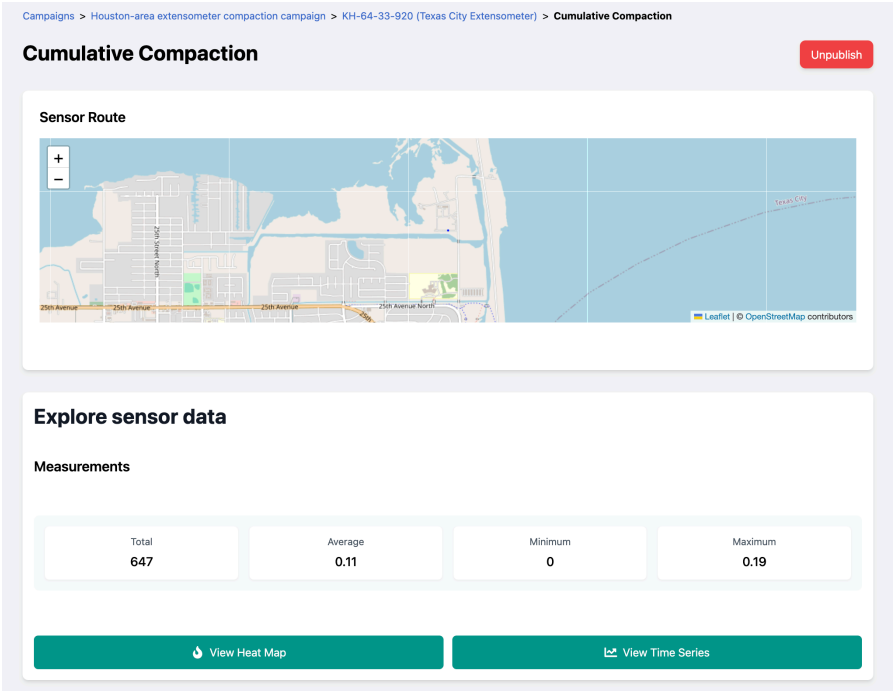
Measurements And Exploration

After data has been uploaded, the UI can be used to review and explore it.

What to learn:

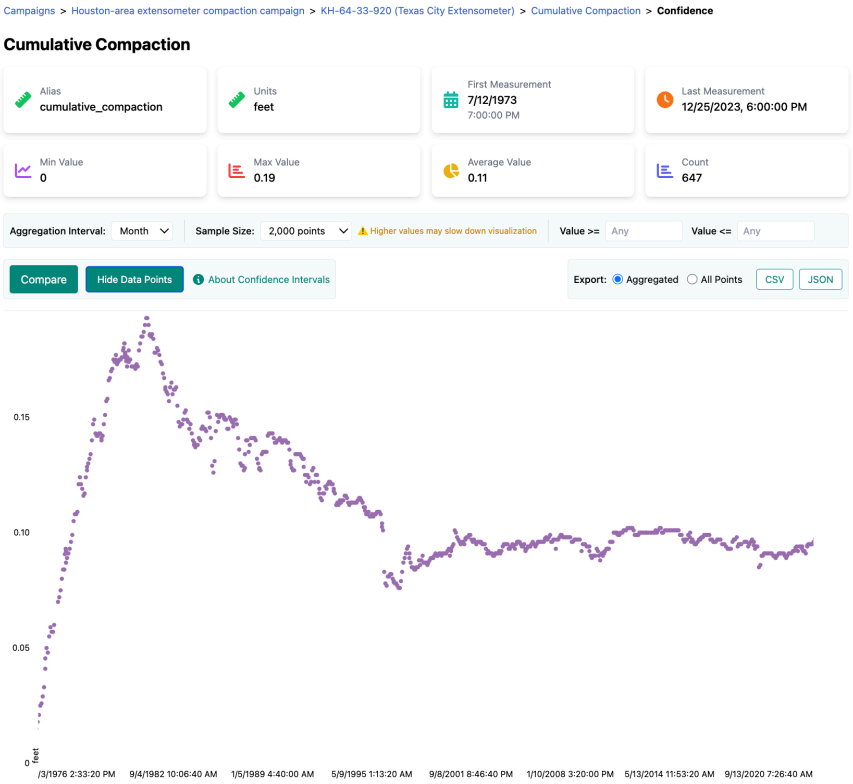
- measurement views help confirm that uploads succeeded

- filtering allows you to focus on a sensor, date range, or subset of observations
- summary or processed views show that the platform supports analysis as well as data management



What to notice in a screenshot:

- timestamps and measurement values
- filters or query controls
- any table, chart, or summary panel that helps interpret the data



Moving From The UI To The SDK

Once you understand the platform structure in the UI, the notebooks are easier to read. The SDK uses the same concepts, but expresses them through Python code.

This is the main reason for using the SDK:

- the UI is useful for guided exploration and manual management
- the SDK is useful for automation, reproducibility, and custom analysis

In other words, the SDK does not replace the UI. It gives you a second way to work with the same platform.

What Notebook 1 Demonstrates

basic-sdk-flow/01_register_campaign_station_upload.ipynb focuses on setup and ingestion.

Before you start:

- make sure you can reach the Upstream service and authenticate from the notebook
- review **data/sensors.csv** and **data/measurements.csv** so you know what will be uploaded
- understand the difference between campaigns, stations, sensors, and measurements in the UI

Expected outputs:

- a campaign and station are created or reused in Upstream
- sensor definitions and time-series measurements are uploaded
- **outputs/flow_context.json** is saved with the campaign and station identifiers used by notebook 2

As you work through it, pay attention to how the notebook:

- authenticates with Upstream
- creates or reuses a campaign
- creates or reuses a station
- validates and uploads **sensors.csv** and **measurements.csv**
- saves campaign and station identifiers for the next notebook

The main lesson in notebook 1 is that a workflow that could be done manually in the UI can also be written as a repeatable Python process.

What Notebook 2 Demonstrates

basic-sdk-flow/02_query_visualize_spatiotemporal.ipynb focuses on retrieval and analysis.

Before you start:

- run notebook 1 first, or have valid campaign and station identifiers available
- confirm that the target station has sensors and uploaded measurements
- keep **outputs/flow_context.json** available if you want the notebook to load the previous context automatically

Expected outputs:

- available sensors are listed for the selected station
- measurements are retrieved into Python for inspection and plotting
- spatial and time-aware visualizations are displayed in the notebook

As you work through it, pay attention to how the notebook:

- reconnects using the saved campaign and station context
- lists available sensors
- selects a sensor and queries measurements
- retrieves GeoJSON for spatial workflows
- builds a spatial map
- builds a time-aware visualization showing how observations change over time

The main lesson in notebook 2 is that data stored in Upstream can be retrieved into Python and used for custom visualization and downstream analysis.

Recommended Order Of Study

To get the most from the tutorials, use the following order:

1. Review the UI and make sure you understand campaigns, stations, sensors, and measurements.

2. Study screenshots of the UI carefully and note how information is grouped.
3. Run notebook 1 and focus on how the SDK performs setup and upload tasks.
4. Run notebook 2 and focus on how the SDK retrieves, filters, and visualizes data.

Most Useful Screenshots

If you are provided with screenshots as part of the tutorial, the most useful ones are:

- a campaign page showing project context
- a station page showing location-specific details
- an upload page showing the CSV workflow
- a measurement view showing timestamps and values

Additional helpful screenshots include:

- a sensor list or sensor detail view
- a statistics or summary view
- a published or shared data view, if available

Main Takeaway

The most important idea to retain is that Upstream presents one consistent data model through two complementary interfaces:

- the UI helps you understand, manage, and inspect data interactively
- the SDK helps you automate, repeat, and extend that work in Python

Learning both views of the platform will make the tutorials more meaningful and will make it easier to understand why the SDK workflow is structured the way it is.